1.模型部署实验手册

1.1 实验部署说明

本实验手册按照“从本地体验到企业部署”的顺序展开。

首先使用 **Ollama** 在本地快速部署一个大模型，让大家理解模型如何被下载、运行，并通过 API 被 Python 程序调用。随后介绍 **vLLM** 在服务器上的部署方式，说明企业环境中为什么更常使用 vLLM。最后使用 **FastAPI** 对模型服务进行二次封装，使其具备更接近真实业务系统的接口形式。

教学定位：

工具	定位	适合场景
Ollama	本地快速体验工具	教学演示、个人电脑测试、快速验证模型
vLLM	高性能推理引擎	服务器部署、高并发 API 服务、企业内部模型服务
FastAPI	业务封装框架	鉴权、日志、限流、Prompt 管理、业务系统接入
OpenAI SDK	统一调用方式	兼容 Ollama、vLLM 和其他 OpenAI 风格服务

一句话总结：

> Ollama 负责让学生快速理解“模型如何跑起来”，vLLM 负责解决“模型如何高效服务化”，FastAPI 负责解决“模型如何接入业务系统”。

1.1.1 Ollama 安装

Ollama 是目前最简单、最适合教学的大模型本地部署工具。下面我们一步一步完成安装。

1. 下载

1. 打开浏览器，访问 Ollama 官方下载页面： https://ollama.com/download
2. 页面会自动识别你的操作系统，点击 **Download for Windows**（Windows 用户）下载安装包 `OllamaSetup.exe`。

注意：下载完成后，请把安装包放在容易找到的位置（如桌面）。

2. 安装

最简单安装方式：

1. 双击下载好的 `OllamaSetup.exe`
2. 一直点击 **“Next”** 或 **“安装”**（全部使用默认设置）
3. 安装完成后，桌面会出现 Ollama 图标，程序会自动启动

注意：默认安装路径在 **C 盘**（推荐不要改动，保持默认即可）。

如果你想自定义安装位置（不太推荐）：

1. 按 `Win + R` 键，输入 `cmd` 打开命令提示符

2. 进入安装包所在目录，例如：

```
cd Desktop
```

3. 执行带路径的安装命令（把路径改成你想要的位置）：

```
OllamaSetup.exe /D=C:\Ollama
```

3. 验证是否安装成功

安装完成后，打开 **PowerShell** 或 **命令提示符**，输入以下命令并回车：

```
ollama --version
```

如果显示类似下面信息，则说明安装成功：

```
E:\Downloads>ollama --version
ollama version is 0.23.4
```

常见问题处理：

- 如果提示“ollama 不是内部或外部命令”，请重启电脑后再试
- 如果还是失败，重新双击 `OllamaSetup.exe` 进行修复安装

1.1.2 开启服务

重要：下面所有操作都在 PowerShell 或 CMD 中完成，不需要打开 Ollama 的图形界面。

1.启动 Ollama 服务（必须先做）

在 **PowerShell** 或 **CMD** 中执行：

```
ollama serve
```

- 这个命令会启动 Ollama 的后台服务。
- 启动成功后，你会看到类似：

```
Listening on [::]:11434
```

（默认端口是 **11434**）

注意：

- 这个窗口**不要关闭**，服务需要一直运行。
- 服务运行期间，其他程序才能调用 Ollama；
- 如果关闭窗口，服务可能停止。

2. 修改监听端口

如果默认端口 `11434` 被占用，可以换一个端口，例如 `11435`：

```
$env:OLLAMA_HOST = "0.0.0.0:11435"; ollama serve
```

参数解释：

- `OLLAMA_HOST`：设置 Ollama 服务监听地址；
- `0.0.0.0`：允许局域网其他设备访问；
- `11435`：新的服务端口；
- `ollama serve`：启动服务。

3. 查看服务是否正常

打开浏览器，访问：

```
http://localhost:11434
```

如果改用了 `11435`，则访问：

```
http://localhost:11435
```

如果页面显示类似：

```
Ollama is running
```

说明服务正常运行。

1.1.3 查看模型列表

服务启动后，再**新开一个 PowerShell 窗口**

1.以下是推荐的 Ollama 模型列表：

模型名称	参数量	大致 RAM 占用（4bit 量化）	推荐用途	推荐指数	适合配置
qwen2.5:3b	3B	**2.5 ~ 3.5 GB**	快速测试、轻量任务	★★★★★	低配电脑（8GB 内存）
qwen2.5:7b	7B	**4.5 ~ 6 GB**	**教学首选、日常使用**	★★★★★	中配电脑（16GB 内存）

模型名称	参数量	大致 RAM 占用（4bit 量化）	推荐用途	推荐指数	适合配置
qwen2.5:14b	14B	**8 ~ 10 GB**	需要更高准确性	★★★★☆	高配电脑（24GB+ 内存）
llama3.2:3b	3B	**2.5 ~ 3.5 GB**	英文任务、轻量对话	★★★★☆	低配电脑
deepseek-r1:7b	7B	**5 ~ 7 GB**	复杂推理、逻辑题	★★★★☆	中高配电脑
qwen2.5:32b	32B	**18 ~ 22 GB**	极致精度（不推荐初学者）	★★★☆☆	高端显卡（40GB+）

2.查看 Ollama 官方提供的全部模型列表（所有可下载的模型）

Ollama 没有直接的命令行查看全部模型列表，最简单的方式是：

打开浏览器访问官网模型库：

🔗 https://ollama.com/library

这个页面会显示 Ollama 官方支持的所有模型，你可以直接搜索、浏览和选择。

上面是点开 deepseek-v3 的详细界面

1.1.4 下载模型

当我们从模型列表里面选择好模型后，直接 run 即可。

1.运行命令示例：

```
# 推荐教学模型（7B）
ollama run qwen2.5:7b

# 轻量测试模型（3B）
ollama run qwen2.5:3b
```

运行的结果：

```
E:\Downloads>ollama run qwen2.5:3b
pulling manifest
pulling 5ee4f07cdb9b: 100% ▕████████████████████▏ 1.9 GB
pulling 66b9ea09bd5b: 100% ▕████████████████████▏   68 B
pulling eb4402837c78: 100% ▕████████████████████▏ 1.5 KB
pulling b5c0e5cf74cf: 100% ▕████████████████████▏ 7.4 KB
pulling 161ddde4c9cd: 100% ▕████████████████████▏  487 B
verifying sha256 digest
writing manifest
success
>>> Send a message (/? for help)
```

2.进入对话模式

你已经成功运行了模型 `qwen2.5:3b`，现在 Ollama 已经处于**对话模式。**

当前界面显示：

```
>>> Send a message (/? for help)
```

这代表：

- 模型已经成功下载并加载
- Ollama 服务正在正常运行
- 你可以直接在终端里输入问题和模型聊天

3.实际操作

在 `>>>` 后面输入任意问题，然后按回车，例如：

```
>>> 介绍一下你自己
当然，很高兴为您介绍自己。

我是来自阿里云的大模型Qwen。我是阿里云自主研发的超大规模语言模型，旨在提供高质量
的对话体验。我的设计目的是在多种任务上超越人类表现，并且在我的训练过程中没有接受
过任何带有偏见的数据集。

我具备广泛的知识面和强大的学习能力，可以帮助您解答问题、撰写代码、创作故事、撰写
邮件、制定营销策略等，帮助您提升生产效率并节省时间。无论您有怎样的需求，我都将竭
诚为您提供服务。

在与您的对话过程中，请记得我的能力目前主要依赖于我所被训练的数据，并且我会不断地
学习以提供更好的帮助。尽管我是一个智能模型，但我仍然需要通过理解和分析上下文来生
成合适的回答，因此我的回答可能会出现一些错误或不准确的地方。对于具体应用场景中的
问题，请您务必进行适当的验证和核实。

如果您有任何关于我的使用方式或者功能的问题，欢迎随时向我提问。我会尽全力帮助您解
决问题。
```

操作提示：

- 输入问题后按 **Enter** 发送
- 输入 `/?` 可以查看帮助命令
- 输入 `/bye` 可以退出当前对话模式

4.结束对话

```
>>> /bye

E:\Downloads>
```

1.1.5 更改下载模型位置

Ollama 默认把模型存在 `C:\Users\你的用户名\.ollama\models`，这个位置通常在 C 盘，很容易把 C 盘占满。

方式一：临时更改（适合测试）

在 PowerShell 中先设置路径，再启动服务：

```
$env:OLLAMA_MODELS = "D:\ollama_models"    # 把 D:\ollama_models 改成你想要的路径
ollama serve
```

注意：

- 这个设置只在当前 PowerShell 窗口有效
- 关闭窗口后就会失效

方式二：永久更改

1. 右键 **此电脑** → **属性** → **高级系统设置** → **环境变量**
2. 在“用户变量”区域点击 **新建**：
    - 变量名：`OLLAMA_MODELS`
    - 变量值：你想要存放模型的路径（例如 `G:\ollama_models`）
3. 点击确定保存
4. **重启 PowerShell**，然后运行：

```
ollama serve
```

3.验证是否修改成功

运行下面命令查看当前模型存储路径：

```
E:\Downloads>ollama list
NAME          ID              SIZE      MODIFIED
qwen2.5:3b    357c53fb659c    1.9 GB    8 minutes ago
```

或者直接打开你设置的文件夹，看是否有 `models` 文件夹生成。

建议：

- 推荐把模型存到 **D 盘或 E 盘**（不要放 C 盘）
- 路径最好简单一点，例如 `D:\ollama_models`
- 修改完成后，第一次运行模型时会重新下载（因为路径变了）

1.1.6 更多模型操作

- `ollama rm qwen2.5:3b` → 删除模型
- `ollama run qwen2.5:3b` → 运行更小的 3B 模型（速度更快）

1.2 暴露 API 服务

当执行 `ollama serve` 后，Ollama 就已经把本地模型变成了一个可以通过 HTTP 访问的 API 服务。

这一步非常重要，因为后续不管是 Agent、Python 程序、网页前端，还是 FastAPI 服务，本质上都是通过 API 调用模型。

1.2.1 什么是暴露 API

可以这样理解：

使用方式	说明
终端对话	在命令行里输入问题，模型直接回答
API 调用	其他程序通过 HTTP 请求调用模型
业务集成	前端、Agent、知识库系统调用模型完成任务

暴露 API 的意义是：

> 模型不再只是一个命令行工具，而是变成了可以被程序调用的智能服务。

1.2.2 启动 API 服务

1. 打开 **PowerShell**
2. 输入以下命令并回车（保持窗口不要关闭）：

```
ollama serve
```

- 默认端口是 **11434**
- 如果你之前修改过端口（如 11435），则使用对应端口

启动成功后，你会看到类似下面信息：

```
Listening on [::]:11434
```

3.验证 API 服务是否正常

打开浏览器，访问以下地址：

http://localhost:11434

如果页面显示 **“Ollama is running”**，就说明 API 服务已经成功暴露，可以被外部程序调用。

1.2.3 快速调用 api

1.快速调用指令

```
curl -X POST "http://localhost:11434/api/generate" -d "
{\"model\":\"qwen2.5:3b\",\"prompt\":\"hello\"}"
```

这条命令的含义是：

- `curl`：发送 HTTP 请求；
- `-X POST`：使用 POST 请求；
- `http://localhost:11434/api/generate`：Ollama 生成接口地址；
- `model`：指定模型；
- `prompt`：输入给模型的问题；
- `stream:false`：一次性返回完整结果。

2.常用 API 接口介绍

接口地址	用途	说明
`/api/generate`	单次生成回复	最常用
`/api/chat`	多轮对话	适合聊天场景
`/api/tags`	查看已下载的模型列表	类似 `ollama list`
`/api/pull`	下载模型	程序自动下载模型

教学小结：

`ollama serve` 一条命令就能把本地模型变成一个 **HTTP API 服务。**

暴露 API 后，我们就可以在 Python 代码、Agent、Multi-Agent 中通过网址调用模型，这是后续课程中 Agent 调用大模型的核心基础。

1.3 requests库调用

`requests` 是 Python 中最常用、最标准的 HTTP 请求库。我们可以使用它直接调用 Ollama 的 API 接口，实现程序与模型的交互。

1.3.1 requests的基本使用方式

安装（Anaconda 通常已自带，如果没有则执行）：

```
pip install requests
```

1.基本使用模板（POST 请求）：

```python
import requests

url = "http://localhost:11434/接口地址"   # Ollama 默认地址

payload = {
    "key1": "value1",
    "key2": "value2"
}

response = requests.post(url, json=payload)   # 发送 JSON 数据

if response.status_code == 200:
    data = response.json()                    # 解析返回的 JSON
    print(data)
else:
    print("请求失败:", response.text)
```

2.post含义

POST 是 HTTP 请求的一种方法，含义是"提交数据"。

- **GET**：从服务器获取数据（像浏览器打开网页）
- **POST**：向服务器发送数据，让服务器处理（像提交表单）

Ollama 的生成接口（/api/generate 和 /api/chat）**必须使用 POST**，因为我们需要把"模型名称 + 提示词"这些数据发送给 Ollama，让它生成回复。

3.常见status_code

HTTP 响应中最常见的几种状态码及其含义：

状态码	含义	说明
200	OK（成功）	请求成功，最常见
400	Bad Request（请求错误）	发送的数据格式有问题
404	Not Found（未找到）	接口地址写错了
500	Internal Server Error	服务器内部错误（Ollama 出问题）

教学小结：在调用 Ollama 时，最重要的是检查 response.status_code 是否为 **200**，如果不是 200，就说明请求失败，需要查看错误信息。

4.response格式解析

当请求成功（status_code == 200）时，response.json() 返回的是一个 Python 字典，常见字段如下：

单次生成（/api/generate）返回格式：

```
{
  "model": "qwen2.5:3b",            # 使用的模型名称
  "created_at": "2026-05-15T...",   # 生成时间
  "response": "这里是模型回复的完整文本",    # 最重要的字段
  "done": true,                     # 是否生成完成
  "total_duration": 1234567890      # 总耗时（纳秒）
}
```

多轮对话（/api/chat）返回格式：

```
{
  "model": "qwen2.5:3b",
  "created_at": "...",
  "message": {
    "role": "assistant",
    "content": "这里是模型回复的完整文本"      # 最重要的字段
  },
  "done": true
}
```

实际使用时最常用的写法：

```
result = response.json()

# 单次生成
print(result["response"])

# 多轮对话
print(result["message"]["content"])
```

教学小结： 调用成功后，最重要的是取出 response 或 message["content"] 这两个字段，这就是模型返回的最终答案。

1.3.2 单次生成

使用 `/api/generate` 接口进行**单次文本生成**（适合简单问答）。

```
import requests

url = "http://localhost:11434/api/generate"

payload = {
    "model": "qwen2.5:3b",                    # 替换为你已下载的模型名称
    "prompt": "请简单介绍一下《中华人民共和国个人信息保护法》的核心内容",
    "stream": False                           # False 表示一次性返回完整结果
}

response = requests.post(url, json=payload)

```

```python
if response.status_code == 200:
    result = response.json()
    print(result["response"])
else:
    print("调用失败:", response.text)
```

1.3.3 多轮对话

使用 `/api/chat` 接口进行**多轮对话**，这是实际应用中最常用的方式。

```python
import requests

url = "http://localhost:11434/api/chat"

payload = {
    "model": "qwen2.5:3b",
    "messages": [
        {"role": "user", "content": "你好，请介绍一下自己"},
        {"role": "assistant", "content": "我是基于 Qwen2.5 的法律助手..."},
        {"role": "user", "content": "请解释一下个人信息保护法第13条"}
    ],
    "stream": False
}

response = requests.post(url, json=payload)

if response.status_code == 200:
    result = response.json()
    print(result["message"]["content"])
else:
    print("调用失败:", response.text)
```

关于记忆存储:

- Ollama **本身不保存对话历史。**
- 每次调用 `/api/chat` 时，必须把完整的对话历史（`messages` 列表）一起发送。
- 记忆是由**你的代码**负责维护的（每次调用前把历史记录追加进去）。

教学小结:

`requests` 是调用 Ollama 最基础、最灵活的方式。掌握单次生成和多轮对话后，你就可以在 Multi-Agent 项目中轻松集成 Ollama 模型。

1.4 使用 ollama 官方包

除了使用 `requests` 直接调用 API，Ollama 官方还提供了一个简洁的 Python 库 `ollama`，使用起来更加方便、直观。

1.4.1 安装

在终端中执行以下命令（只需安装一次）：

```
pip install ollama
```

1.4.2 单次生成

使用 `ollama.generate()` 进行单次文本生成：

```
import ollama

response = ollama.generate(
    model="qwen2.5:3b",                          # 替换为你已下载的模型名称
    prompt="请简单介绍一下《中华人民共和国个人信息保护法》的核心内容"
)

print(response['response'])
```

1.4.3 多轮对话

使用 `ollama.chat()` 进行多轮对话（推荐方式）：

```
import ollama

response = ollama.chat(
    model="qwen2.5:3b",
    messages=[
        {"role": "user", "content": "你好，请介绍一下自己"},
        {"role": "user", "content": "请解释一下个人信息保护法第13条的内容"}
    ]
)

print(response['message']['content'])
```

1.4.4 与 requests 的对比

对比维度	requests 方式	ollama 官方包方式	推荐场景
代码简洁度	较繁琐	非常简洁	ollama 包更推荐
易用性	需要手动处理 JSON	自动处理	ollama 包更友好
灵活性	最高	稍低	requests 更灵活
学习成本	稍高	低	初学者推荐 ollama 包

教学小结：

`ollama` 官方包是调用 Ollama 最简洁的方式，适合快速开发和教学。

`requests` 方式更底层，适合需要完全自定义请求头的场景。

1.5 OpenAI 兼容调用与常见问题

Ollama 提供了 OpenAI 兼容接口，我们可以使用官方 `openai` 库来调用 Ollama，代码风格和调用 ChatGPT 一致。

1.5.1 调用方式

```python
from openai import OpenAI

client = OpenAI(
    base_url="http://localhost:11434/v1",    # Ollama 的 OpenAI 兼容地址
    api_key="ollama"                         # Ollama 不需要真实 key，随便填
)

response = client.chat.completions.create(
    model="qwen2.5:3b",
    messages=[
        {"role": "user", "content": "请简单介绍一下《中华人民共和国个人信息保护法》的核心内容"}
    ]
)

print(response.choices[0].message.content)
```

1.5.2 ChatOpenAI 的支持性

`langchain_openai.ChatOpenAI` 完全兼容 Ollama，只需修改 `base_url` 即可：

```python
from langchain_openai import ChatOpenAI

llm = ChatOpenAI(
    model="qwen2.5:7b",
    base_url="http://localhost:11434/v1",
    api_key="ollama"           # 随便填
)

response = llm.invoke("请介绍一下个人信息保护法第13条")
print(response.content)
```

教学小结：

使用 OpenAI 兼容方式调用 Ollama 是目前最推荐的方法，既可以直接用 `openai` 库，也可以在 LangChain 中通过 `ChatOpenAI` 使用，代码切换成本极低！

1.5.3 远程调用 API

默认情况下，Ollama 只允许**本机（localhost）**访问。如果希望其他电脑（或局域网内的其他设备）也能调用你的 Ollama 服务，需要进行以下设置：

操作方法：

1. 启动服务时指定监听所有 IP：

```
$env:OLLAMA_HOST = "0.0.0.0:11434"; ollama serve
```

2. 在其他电脑上通过你的 IP 地址访问，例如：

```
http://192.168.1.100:11434
```

注意事项：

- 局域网内其他设备必须知道你的 IP 地址（可在命令提示符输入 `ipconfig` 查看）
- 防火墙需要放行 11434 端口
- **生产环境不推荐直接暴露**，存在安全风险

1.5.4 接口安全说明

Ollama 默认不提供完善的生产级鉴权机制。也就是说，如果你把 Ollama 服务暴露到局域网或公网，其他人只要知道地址，就可能直接调用你的模型。

因此：

- 本地学习时可以直接使用；
- 局域网测试时要谨慎开放端口；
- 生产环境不推荐直接暴露 Ollama；
- 正式项目中建议使用 FastAPI、Nginx 或 API Gateway 在外层增加鉴权、限流和访问控制。

教学小结：

Ollama 适合本地学习和小范围测试，不适合作为直接暴露的生产 API。后续我们会使用 FastAPI 对模型服务进行封装，在业务层实现更规范的鉴权和访问控制。

1.5.5 其他常见问题

在使用 Ollama 过程中，经常会遇到以下问题，以下是常见问题及解决方案：

问题 1：模型下载速度很慢

原因：国内网络访问 Hugging Face / Ollama 官方源较慢。

解决方案：

- 使用镜像源（推荐）：

```
$env:OLLAMA_HOST = "0.0.0.0:11434"
$env:OLLAMA_MODELS = "D:\ollama_models"
ollama serve
```

- 或提前使用浏览器手动下载模型文件再放入对应文件夹。

问题 2：显存不足（Out of Memory）

原因：模型太大，电脑显存不够。

解决方案：

- 换用更小的模型（如 `qwen2.5:3b` 或 `qwen2.5:7b`）
- 使用量化版本（Ollama 默认已使用量化）
- 关闭其他占用显存的程序

问题 3：端口被占用

原因：之前启动的服务没有正常关闭。

解决方案：

```
Get-Process -Id (Get-NetTCPConnection -LocalPort 11434).OwningProcess | Stop-Process -Force
```

问题 4：中文回复效果差

原因：使用了英文优化模型。

解决方案：

- 优先使用中文优化模型：`qwen2.5:7b`、`qwen2.5:14b`
- 在 prompt 中明确要求："请用中文回复"

问题 5：服务启动后浏览器访问 404

原因：端口不正确或服务未启动。

解决方案：

- 确认 `ollama serve` 窗口仍在运行
- 使用正确的端口（默认 11434）

教学小结：

Ollama 使用过程中最常见的问题主要是**网络、端口、显存**三类。遇到问题时，先检查 `ollama serve` 是否在运行，再检查端口，最后根据错误提示搜索解决方案。

2. 从本地实验到企业部署优化

在第 1 部分中，我们已经使用 **Ollama** 完成了本地模型部署、终端对话、API 暴露和 Python 调用。对于 Windows 本地教学环境来说，Ollama 依然是最合适的实验工具，因为它安装简单、模型下载方便，并且能稳定提供本地 API 服务。

但是，在企业生产环境中，大模型通常不会直接部署在个人 Windows 电脑上，而是部署在 **Linux GPU 服务器**上。此时，系统更关注并发能力、显存利用率、吞吐量、稳定性和可扩展性，因此通常会使用 **vLLM** 这类高性能推理引擎。

因此，本部分的目标不是否定 Ollama，而是说明：

> **Ollama 负责本地实验跑通，vLLM 负责企业服务器优化，FastAPI 负责统一封装模型服务。**

整体路径如下：

```
Windows 本地实验
        ↓
Ollama 跑通模型和 API 调用
        ↓
FastAPI 封装本地 Ollama 服务
        ↓
理解业务系统如何调用大模型
        ↓
企业服务器环境中使用 vLLM 替换底层推理服务
```

2.1 本地实验与企业部署的区别

本地实验和企业部署的目标不同，不能简单用同一套工具解决所有问题。

在课程实验阶段，我们优先保证学生能够在自己的电脑上跑通完整流程；在企业生产阶段，则更关注性能、稳定性和扩展能力。

2.1.1 本地实验为什么继续使用 Ollama

Ollama 非常适合本地教学实验，原因包括：

- 支持 Windows；
- 安装简单；
- 一条命令即可下载并运行模型；
- 可以直接暴露 HTTP API；
- 支持 OpenAI 兼容接口；
- 适合学生在个人电脑上完成实验。

对于教学来说，最重要的是让学生先理解：

- 模型如何下载；
- 模型如何运行；

- API 如何调用；
- Python 程序如何和模型交互；
- FastAPI 如何封装模型服务。

这些内容用 Ollama 完全可以完成。

教学结论：

> 本课程的本地实验阶段，仍然以 Ollama 为主。

2.1.2 企业部署为什么引入 vLLM

vLLM 更适合部署在 Linux GPU 服务器上，用来支撑企业级模型服务。

它相比 Ollama 的主要优势是：

- 更高并发；
- 更高吞吐；
- 更好的 GPU 显存利用率；
- 更适合多用户同时访问；
- 更适合与 Docker、Kubernetes、Nginx 等生产组件结合。

但是，vLLM 对环境要求更高：

- 推荐 Linux 系统；
- 需要 NVIDIA GPU；
- 需要正确配置 CUDA、驱动和 PyTorch；
- Windows 本地环境通常不作为推荐部署方式。

所以在课程中，vLLM 更适合作为：

- 企业生产环境方案；
- 有 GPU 服务器条件的进阶实验。

教学结论：

> vLLM 不是用来替代学生本地 Ollama 实验的，而是用来说明企业环境中如何优化部署。

2.1.3 Ollama 与 vLLM 的正确关系

维度	Ollama	vLLM
课程定位	本地实验主方案	企业部署优化方案
推荐环境	Windows / macOS / Linux 本地电脑	Linux GPU 服务器
上手难度	低	中等偏高
是否适合学生本地实验	非常适合	不适合普通 Windows 本地实验

维度	Ollama	vLLM
是否适合生产高并发	一般	更适合
API 能力	支持 HTTP API 和 OpenAI 兼容接口	支持 OpenAI 兼容接口
FastAPI 封装	可以封装	可以封装
推荐用途	教学、验证、个人开发	企业服务、服务器推理、高并发场景

一句话总结：

> 课堂实验用 Ollama 跑通链路，企业部署用 vLLM 提升性能。

2.2 FastAPI 封装思路

FastAPI 是本部分最关键的桥梁。因为无论底层模型服务是 Ollama 还是 vLLM，FastAPI 都可以用统一方式进行封装。

也就是说，FastAPI 上层接口可以保持不变，底层模型服务可以根据环境切换：

```
本地实验环境：
用户 / 前端 / Python
        ↓
FastAPI
        ↓
Ollama
        ↓
本地模型
```

```
企业服务器环境：
用户 / 前端 / Agent
        ↓
FastAPI
        ↓
vLLM
        ↓
GPU 服务器上的大模型
```

这样设计的好处是：

- 学生本地可以完整跑通；
- 企业部署时只需要替换底层 `base_url`；
- 上层业务接口基本不变；
- 后续接入 Agent、RAG、前端都更方便。

2.3 使用 FastAPI 封装 Ollama 本地服务

本节先完成一个能在 Windows 本地跑通的实验：**FastAPI 调用 Ollama。**

前提条件：

1. 已经安装 Ollama；
2. 已经启动 Ollama 服务；
3. 已经下载并运行模型，例如 `qwen2.5:3b`；
4. Ollama 服务地址为：

```
http://localhost:11434/v1
```

2.3.1 安装依赖

在终端中执行：

```
pip install fastapi uvicorn openai
```

其中：

- `fastapi`：用于编写 Web API；
- `uvicorn`：用于启动 FastAPI 服务；
- `openai`：用于调用 Ollama 的 OpenAI 兼容接口。

2.3.2 编写 FastAPI 服务

新建文件：

```
app_ollama.py
```

代码如下：

```python
from fastapi import FastAPI, Header, HTTPException
from pydantic import BaseModel
from openai import OpenAI

app = FastAPI(title="Local Ollama LLM API")

client = OpenAI(
    base_url="http://localhost:11434/v1",
    api_key="ollama"
)

API_KEY = "your-secret-key"

```

```python
class ChatRequest(BaseModel):
    query: str

@app.post("/chat")
def chat(request: ChatRequest, authorization: str = Header(default="")):
    if authorization != f"Bearer {API_KEY}":
        raise HTTPException(status_code=401, detail="Unauthorized")

    response = client.chat.completions.create(
        model="qwen2.5:3b",
        messages=[
            {"role": "system", "content": "你是一个专业、准确、简洁的中文助手。"},
            {"role": "user", "content": request.query}
        ],
        temperature=0.7
    )

    return {
        "answer": response.choices[0].message.content
    }
```

这段代码完成了三件事：

1. 对外提供 `/chat` 接口；
2. 通过 OpenAI SDK 调用本地 Ollama；
3. 增加了一个简单的 API Key 校验。

2.3.3 启动 FastAPI 服务

在 `app_ollama.py` 所在目录执行：

```
uvicorn app_ollama:app --host 0.0.0.0 --port 9000
```

启动成功后，可以访问：

```
http://localhost:9000/docs
```

FastAPI 会自动生成接口文档页面。

2.3.4 调用 FastAPI 接口

使用 curl 调用：

```
curl -X POST "http://localhost:9000/chat" ^
  -H "Content-Type: application/json" ^
  -H "Authorization: Bearer your-secret-key" ^
  -d "{\"query\":\"请介绍一下大模型部署的基本流程\"}"
```

如果是在 Linux 或 macOS 终端中，可以使用：

```
curl -X POST "http://localhost:9000/chat" \
  -H "Content-Type: application/json" \
  -H "Authorization: Bearer your-secret-key" \
  -d "{\"query\":\"请介绍一下大模型部署的基本流程\"}"
```

完整调用链路如下：

```
curl / 浏览器 / Python
        ↓
FastAPI /chat 接口
        ↓
OpenAI SDK
        ↓
Ollama OpenAI 兼容接口
        ↓
qwen2.5:3b 本地模型
```

教学小结：

这一步非常重要，因为它让学生在 Windows 本地就能完成“模型服务封装”的完整实验。

2.4 FastAPI 封装后的意义

第 1 部分中，我们已经可以直接调用 Ollama，为什么还要再套一层 FastAPI?

原因是：**Ollama 或 vLLM 负责模型推理，FastAPI 负责业务逻辑。**

2.4.1 直接调用模型服务的问题

如果业务系统直接调用 Ollama 或 vLLM，会有一些问题：

- 不方便统一鉴权；
- 不方便做限流；
- 不方便记录业务日志；
- 不方便统一管理 Prompt；
- 不方便接入数据库；
- 不方便接入 RAG 或 Agent；
- 不方便做异常处理；
- 不方便未来切换模型服务。

因此，正式项目中通常不会让前端直接调用底层模型服务，而是通过 FastAPI 这样的业务服务进行封装。

2.4.2 FastAPI 的作用

能力	说明
鉴权	判断请求是否有权限访问模型
日志	记录用户请求、响应耗时、错误信息
限流	防止单个用户过度占用模型资源
Prompt 管理	统一设置 system prompt 和模板
业务逻辑	根据用户、任务类型选择不同处理流程
异常处理	返回统一格式的错误信息
模型切换	底层从 Ollama 切换到 vLLM，上层接口不变

一句话总结：

> FastAPI 让模型从“推理工具”变成“业务服务”。

2.5 从 Ollama 切换到 vLLM

当我们在本地使用 Ollama + FastAPI 跑通后，如果将来部署到企业 GPU 服务器，只需要把底层模型服务从 Ollama 换成 vLLM。

上层 FastAPI 的整体结构可以保持不变，主要修改两个地方：

1. `base_url`
2. `model`

2.5.1 Ollama 与 vLLM 调用差异

服务	base_url	model 示例	适用环境
Ollama	`http://localhost:11434/v1`	`qwen2.5:3b`	Windows 本地实验
vLLM	`http://服务器IP:8000/v1`	`qwen2.5-7b`	Linux GPU 服务器

也就是说，本地实验代码中：

```
client = OpenAI(
    base_url="http://localhost:11434/v1",
    api_key="ollama"
)
```

企业服务器部署时可以改成：

```
client = OpenAI(
    base_url="http://服务器IP:8000/v1",
    api_key="EMPTY"
)
```

模型名也从：

```
model="qwen2.5:3b"
```

改成：

```
model="qwen2.5-7b"
```

2.5.2 FastAPI 封装 vLLM 示例

如果 vLLM 已经在服务器上启动，例如服务地址为：

```
http://192.168.1.100:8000/v1
```

则 FastAPI 代码可以改成：

```
from fastapi import FastAPI, Header, HTTPException
from pydantic import BaseModel
from openai import OpenAI

app = FastAPI(title="Enterprise vLLM API")

client = OpenAI(
    base_url="http://192.168.1.100:8000/v1",
    api_key="EMPTY"
)

API_KEY = "your-secret-key"

class ChatRequest(BaseModel):
    query: str

@app.post("/chat")
def chat(request: ChatRequest, authorization: str = Header(default="")):
    if authorization != f"Bearer {API_KEY}":
        raise HTTPException(status_code=401, detail="Unauthorized")

    response = client.chat.completions.create(
        model="qwen2.5-7b",
        messages=[
            {"role": "system", "content": "你是一个专业、准确、简洁的中文助手。"},
            {"role": "user", "content": request.query}
        ],
        temperature=0.7
```

```
    )

    return {
        "answer": response.choices[0].message.content
    }
```

可以看到，整体代码结构和封装 Ollama 时几乎一样。

变化的只是：

```
base_url="http://192.168.1.100:8000/v1"
```

以及：

```
model="qwen2.5-7b"
```

教学小结：

> FastAPI 层保持稳定，底层模型服务可以从 Ollama 平滑切换到 vLLM。

*2.6 vLLM 服务器部署说明

这一节用于说明企业服务器上如何部署 vLLM。由于 vLLM 更适合 Linux GPU 服务器，因此这部分可以作为教师演示或服务器环境实验。

2.6.1 推荐环境

环境	推荐配置
操作系统	Ubuntu 20.04 / 22.04
Python	3.10 / 3.11
GPU	NVIDIA GPU
驱动	能正常执行 `nvidia-smi`
CUDA	与 PyTorch / vLLM 版本匹配
模型格式	HuggingFace 格式模型

验证 GPU：

```
nvidia-smi
```

如果能看到 GPU 型号、显存和驱动信息，说明 GPU 环境基本正常。

2.6.2 推荐模型

模型名称	参数量	推荐显存	适合场景
Qwen/Qwen2.5-0.5B-Instruct	0.5B	2GB+	接口连通性验证
Qwen/Qwen2.5-1.5B-Instruct	1.5B	4GB+	轻量教学、快速演示
Qwen/Qwen2.5-3B-Instruct	3B	8GB+	教学测试、轻量问答
Qwen/Qwen2.5-7B-Instruct	7B	16GB+	教学首选、常见单卡部署
Qwen/Qwen2.5-14B-Instruct	14B	24GB+	更高质量问答
Qwen/Qwen2.5-32B-Instruct	32B	48GB+	高质量业务场景

课堂演示如果服务器显存充足，可以使用：

```
Qwen/Qwen2.5-7B-Instruct
```

如果只是验证 vLLM 服务是否能启动，可以使用：

```
Qwen/Qwen2.5-0.5B-Instruct
```

2.6.3 安装 vLLM

创建环境：

```
conda create -n vllm python=3.10 -y
conda activate vllm
```

安装 vLLM：

```
pip install vllm
```

验证安装：

```
python -c "import vllm; print(vllm.__version__)"
```

2.6.4 下载模型

安装下载工具：

```
pip install huggingface_hub
```

创建模型目录：

```
mkdir -p /data/models
```

下载模型：

```
hf download Qwen/Qwen2.5-7B-Instruct \
  --local-dir /data/models/Qwen2.5-7B-Instruct
```

如果使用 0.5B 模型进行测试，可以改成：

```
huggingface-cli download Qwen/Qwen2.5-0.5B-Instruct \
  --local-dir /data/models/Qwen2.5-0.5B-Instruct
```

2.6.5 启动 vLLM 服务

以 7B 模型为例：

```
vllm serve /data/models/Qwen2.5-7B-Instruct \
  --host 0.0.0.0 \
  --port 8000 \
  --dtype auto \
  --max-model-len 4096 \
  --gpu-memory-utilization 0.9 \
  --served-model-name qwen2.5-7b
```

如果使用 0.5B 模型测试：

```
vllm serve /data/models/Qwen2.5-0.5B-Instruct \
  --host 0.0.0.0 \
  --port 8000 \
  --dtype auto \
  --max-model-len 2048 \
  --served-model-name qwen2.5-0.5b
```

启动成功后，vLLM 服务地址为：

```
http://服务器IP:8000/v1
```

2.6.6 测试 vLLM 服务

使用 curl 测试：

```bash
curl http://服务器IP:8000/v1/chat/completions \
  -H "Content-Type: application/json" \
  -d '{
    "model": "qwen2.5-7b",
    "messages": [
      {"role": "user", "content": "请介绍一下 vLLM 的核心优势"}
    ],
    "temperature": 0.7
  }'
```

如果使用 0.5B 模型，则 `model` 改为：

```
"model": "qwen2.5-0.5b"
```